

CyberBrass

A single-player cyberpunk FPS — Software Design Document

Document	Software Design Proposal
Status	Draft — For Reviewer Comment
Engine	Unity 6 LTS (HDRP)
Language	C# 12
Persistence	SQLite (sqlite-net-pcl)
Target Platforms	Windows, Linux, macOS
Genre	First-person shooter, single-player
Art Direction	Cyberpunk + dieselpunk; brass-forward palette

Purpose of this document. *This is an early-stage design proposal intended for reviewer comment before a rapid prototype begins. Reviewers are invited to challenge the engine choice, architecture, persistence approach, scope of the first vertical slice, and art direction. Open questions are listed at the end.*

1. Executive Summary

CyberBrass is a single-player first-person shooter set in a brass-and-neon cyberpunk world. The game is built in Unity 6 with C# scripts, uses the High Definition Render Pipeline (HDRP) for film-grade visuals, and persists player progress in a local SQLite database. It targets Windows, Linux, and macOS desktop, distributed as standalone builds.

An earlier iteration of this design proposed a Flutter client driven by a localhost C# API. That direction was rejected after evaluation: Flutter is a 2D UI framework without a production-grade 3D pipeline, and routing single-player game state through an out-of-process API introduces latency that conflicts with a 60+ fps shooter loop. The current design keeps everything in a single Unity process, which is the standard, well-supported approach for this genre.

2. Goals and Non-Goals

Goals

- Visually stunning cyberpunk presentation with prominent brass materials.
- Cross-platform desktop: Windows, Linux, macOS.
- Solid, responsive FPS feel — the controller must feel right before art.
- Local persistence of player profile, loadouts, unlocks, and settings.
- Modular weapon and enemy systems that allow new content via data, not code.
- Shippable in vertical slices so progress is visible at every milestone.

Non-Goals (initial release)

- Multiplayer or networked play.
- Mobile or console platforms.
- Procedurally generated levels.
- Modding support exposed to end users.
- Cloud save synchronization.

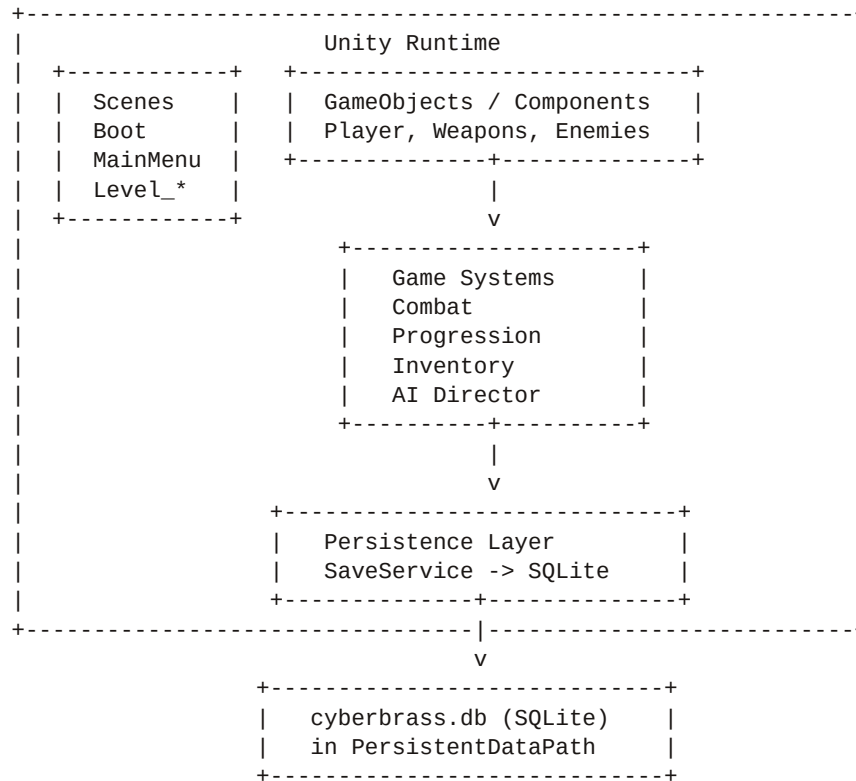
3. Technology Stack

Layer	Choice	Rationale
Engine	Unity 6 LTS	C#-native, mature FPS tooling, three-platform desktop support.
Render Pipeline	HDRP (URP fallback on macOS)	PBR, real reflections, volumetrics — essential for brass.
Language	C# 12	Strong typing, async, large ecosystem, team familiarity.
Database	SQLite via sqlite-net-pcl	Embedded, zero-config, identical behavior across platforms.
Input	Unity Input System	Rebindable bindings, gamepad + KB/M abstraction.
Physics	Built-in PhysX	Sufficient for FPS; no extra dependency.

AI	NavMesh + Behavior package	First-party, well-documented.
Audio	Unity Audio (FMOD optional)	Native support; FMOD only if adaptive music required.
Build / CI	GitHub Actions + game-ci/unity-builder	Reproducible builds for all three targets from a tag.
Version Control	Git + Git LFS	Standard; LFS handles binary art assets.

4. High-Level Architecture

Single-process design. Unity hosts scenes and GameObjects, which compose plain C# systems for game logic. A persistence layer wraps SQLite and exposes async save and load operations to gameplay code.



5. Project Structure

```

/CyberBrass
/Assets
/Art          -- models, textures, materials, VFX
/Audio        -- music, SFX, voice
/Prefabs      -- player, weapons, enemies, props
/Scenes       -- Boot, MainMenu, Level_Undercity, ...
/Scripts
/Core         -- GameManager, SceneLoader, ServiceLocator
/Player       -- PlayerController, CameraRig, FpsLook
/Weapons      -- WeaponBase, Revolver, PneumaticRifle, ...
/Enemies      -- EnemyBase, AI behaviors, spawners
/Combat       -- DamageSystem, HitscanService, Projectile
/UI           -- HUD, menus, settings
/Persistence  -- SaveService, models, SQLite repo
/Progression  -- loadouts, unlocks, currency
/Audio        -- AudioRouter, MusicDirector
/Settings     -- HDRP assets, input maps, render configs
/Packages
/ProjectSettings
  
```

6. Core Systems

6.1 Player and Camera

A CharacterController-based FPS rig with decoupled look (camera) and move (controller). Head-bob, weapon-sway, and recoil are driven by a mix of Animator clips and procedural code. Configurable FOV, sensitivity, and invert-Y. The controller is the first thing to feel right; everything else is built on it.

6.2 Weapons

Each weapon is a ScriptableObject definition (damage, fire rate, spread, ammo type, model, sound bank) paired with a runtime behavior script on the prefab. New weapons and variants are added by authoring data assets — no code changes required.

Launch roster (concept)

- **The Foreman** — brass-cased revolver. Six shots, devastating, slow reload.
- **Sparrowhawk** — pneumatic flechette rifle. Full-auto; brass exhaust ports vent steam.
- **Ohm-Cutter** — arc emitter. Chains between enemies; copper coils glow.
- **Brasswork** — quad-barrel shotgun. Hand-engraved; deafening.
- **Telegraph** — coilgun. Long-range sniper role; brass rails.

6.3 Enemies and AI

EnemyBase plus per-archetype behavior trees. Archetypes: Sentinel (patrol/engage), Stalker (flank), Brute (rush), Drone (aerial). Pathing via NavMesh, perception via sphere casts and line-of-sight. An AIDirector regulates spawn pressure based on player performance to keep tension dynamic without scripting every encounter.

6.4 Combat

DamageSystem operates as an event bus: weapons raise DamageEvents, targets subscribe and resolve them. Hitscan via raycasts; projectiles for slower weapons. Death triggers ragdoll using Unity's built-in joints. Optional damage numbers and hit markers, toggleable in accessibility settings.

6.5 Persistence (SQLite)

An ISaveRepository interface abstracts storage; SqliteSaveRepository is the concrete implementation. SaveService is the façade gameplay code talks to. Writes are async and debounced — not every kill flushes to disk. Flushes occur at checkpoints, level end, pause, and quit.

Table	Columns (abbreviated)
PlayerProfile	id, name, created_at, total_playtime, currency
Loadouts	id, profile_id, slot, primary_weapon, secondary_weapon, mods_json
Unlocks	profile_id, item_id, unlocked_at
Settings	profile_id, key, value (flexible key/value)
RunHistory	id, profile_id, level, duration, kills, deaths, completed_at

DB path: `Application.persistentDataPath/cyberbrass.db`

6.6 UI and HUD

UI Toolkit (USS/UXML — feels like web) drives menus and HUD. The HUD carries the art direction: brass bezels, an analog ammo dial with a digital overlay, a vacuum-tube health indicator whose filament dims with damage, a compass strip with objective markers, and HDRP post-processing for vignette and chromatic aberration on low health.

6.7 Audio

A MusicDirector swaps between exploration, combat, and boss states with crossfades. Diegetic audio is prioritized — buzzing neon, brass shell casings on concrete, distant steam vents — to sell the world.

7. Art Direction

Brass-forward cyberpunk. The HDRP pipeline was chosen specifically because brass demands accurate reflections and PBR materials to read as expensive rather than gaudy.

Palette

Color	Name	Hex
	Aged brass	#B5824A
	Oxidized copper / verdigris	#3E7C6A
	Soot black	#1A1614
	Amber CRT glow	#FFB347
	Magenta neon	#FF2E88
	Deep teal	#1E5F74

Materials and props

- Scratched brass plating with verdigris in recesses.
- Etched copper circuit boards used as architectural cladding.
- Smoked glass, exposed filament tubes, vacuum-tube readouts.
- Riveted brass armor over synthetic muscle on enemies.
- Brass-cased weapons with holographic sights and analog dials.

Environments

- **The Undercity** — rain-slick alleys, brass pipework, neon signage in Cyrillic and kanji.
- **The Foundry** — active brass smelters; environmental hazards.
- **The Spire** — corporate vertical level; polished brass and glass.

8. Cross-Platform Considerations

- **Windows** — primary development target. DirectX 12.
- **Linux** — Vulkan graphics API; build via game-ci.
- **macOS** — Metal. HDRP works but consider URP fallback for older hardware.
- **SQLite** — sqlite-net-pcl bundles native binaries for all three.
- **Input** — Unity Input System normalizes gamepad differences automatically.
- **CI** — GitHub Actions matrix builds all three targets from a tagged commit.

9. Build Order (Milestones)

Each milestone is independently shippable in the sense that it produces a playable demo of some kind. This avoids the trap of building for a year and discovering the controller does not feel right.

#	Milestone	Exit Criteria
1	Greybox vertical slice	Player walks, jumps, shoots hitscan at static targets in a blockout. No art.
2	Combat loop	One enemy archetype with NavMesh AI, damage, death, basic HUD.
3	Persistence	SQLite saves profile, settings, currency. Saves on quit; loads on launch.
4	Art bar	One weapon and one enemy fully art-passed. Establishes the visual target.
5	First real level	The Undercity, end-to-end, with audio.
6	Progression	Unlocks, loadouts, currency loop.
7	Polish + content	Additional levels, weapons, accessibility pass, balancing.

10. Risks and Mitigations

Risk	Mitigation
HDRP performance on macOS	Provide a URP-fallback render path; ship a 'Performance' graphics preset.
Art pipeline complexity (PBR brass)	Establish art bar early (Milestone 4) before producing full content.
AI Director tuning	Telemetry from RunHistory drives data-informed difficulty curves.
Scope creep (multiplayer, mods)	Explicit non-goals in Section 2; revisit only after launch.
Single developer bandwidth	Vertical-slice milestones keep the project demoable at all times.
Input System gotchas across platforms	Continuous integration build smoke-tests input on all three OSes.

11. Open Questions for Reviewers

- Is Unity 6 + HDRP the right call, or should we evaluate Unreal 5 for higher-end visuals?
- Should SQLite be wrapped behind a generic repository now, or only if we ever need cloud sync?
- Is the proposed weapon roster (5 weapons) the right launch count, or should we ship with 3 and add via updates?
- Should we target gamepad-first or KB/M-first for the early prototype's feel testing?
- Is HDRP-on-macOS via Metal acceptable, or should macOS be deprioritized until later?
- Should an art-direction prototype (Milestone 4) precede the combat loop (Milestone 2)?
- Are the listed non-goals correct, or should any be promoted to goals?

Reviewers: please leave comments inline (highlight + comment) or in a separate review document referencing section numbers. The next step after review is a one-week rapid prototype targeting Milestone 1 only.